

Perfect. This is exactly the state we wanted.

Your output proves three important things:

```
Service: spring-app
Port: http 8080/HTTP targets pod port 8080

RBAC policies:
  spring-workload-authorization-rule[0]
  spring-workload-authorization-rule[1]

Effective PeerAuthentication:
  Workload mTLS mode: STRICT

Applied PeerAuthentication:
  demo-strict-mtls.demo
```

So for your successful request:

Bash

```
kubectl exec -n service-a deploy/service-a -c service-a -- \
  curl -i http://spring-app.demo.svc.cluster.local:8080/api/internal
```

we now know:

```
service-a
|
| request enters Service A Envoy
▼
Envoy sidecar
|
| mTLS
| client workload identity
| service-a/service-a
▼
Spring Envoy
|
|─ PeerAuthentication: STRICT ✓
|
|─ AuthorizationPolicy applied ✓
|
▼
Spring Boot
/api/internal
|
▼
200 OK
```

In particular, because Spring is `STRICT`, its Envoy will not accept plaintext traffic for that workload. Your successful sidecar-to-sidecar request therefore crossed the mesh connection using mTLS.

Now prove the authorization

This is the interesting part of the POC.

Your current successful client has:

```
namespace      service-a
ServiceAccount service-a
```

Now create another pod in the **same namespace** but let it use `default`:

Bash

```
kubectl run bad-client \
  -n service-a \
  --image=curlimages/curl:8.12.1 \
  --restart=Never \
  --command -- sleep 3600
```

Wait:

Bash

```
kubectl get pod bad-client -n service-a
```

Make sure it's:

```
2/2 Running
```

That `2/2` is important because we want:

```
curl container + istio-proxy
```

Confirm the ServiceAccount:

Bash

```
kubectl get pod bad-client -n service-a \
  -o jsonpath='{.spec.serviceAccountName}{"\n"}'
```

Expected:

default

Now call exactly the same endpoint:

Bash

```
kubectl exec -n service-a bad-client -c bad-client -- \
  curl -i \
  http://spring-app.demo.svc.cluster.local:8080/api/internal
```

We want:

HTTP/1.1 403 Forbidden

RBAC: access denied

If that happens, you've demonstrated a very clean separation:

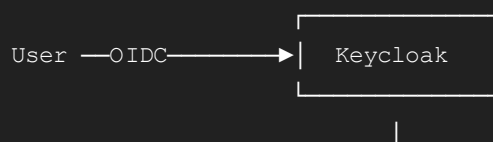


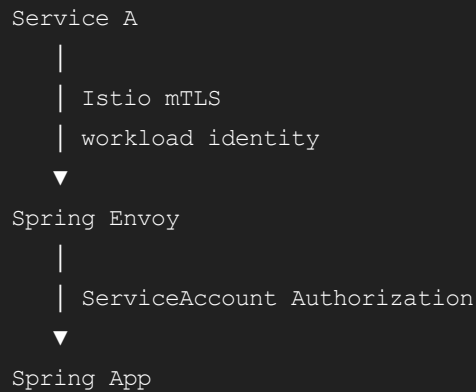
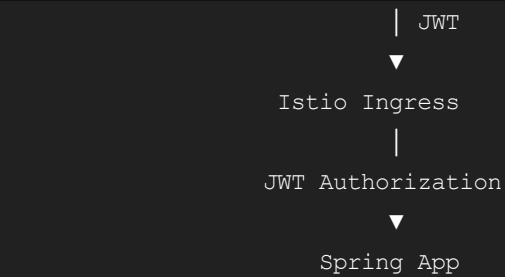
That's an excellent demo of an important concept:

mTLS authentication answers "Who is calling?" while AuthorizationPolicy answers "Is that identity allowed to do this?"

And notice that **Keycloak isn't involved at all** in this particular flow. That's intentional.

Your architecture is starting to show three distinct identity/security mechanisms:





After we get **bad-client = 403**, the next experiment I'd do is especially useful for your presentation: make Service A a **Keycloak OAuth client using Client Credentials**.

Then Service A can present **two identities simultaneously**:

```
mTLS:
cluster.local/ns/service-a/sa/service-a
    ↓
"Which Kubernetes workload is calling?"

Keycloak JWT:
client_id = service-a
roles = [...]
    ↓
"Which OAuth client/application is calling?"
```

Then we can build an Istio policy requiring **both**. That will make the distinction between Istio workload identity and Keycloak/OAuth application identity very clear.